DATAPULSE

Build Plan

Phased roadmap, learning notes and senior full-stack interview preparation

Version 1.0 | 12 October 2026

What this document is

A working plan for building DataPulse step by step, written so that every step is also a lesson. Each phase explains what we build, why it matters, which concepts you should be able to explain, and how we know the phase is done. The final section is an interview bank of senior full-stack questions that spans every part of the system, with answers grounded in decisions made in this project.

How to use it

- Work through the phases in order. Each one leaves the repository in a working state.
- Before starting a phase, read its Concepts list. After finishing, explain each concept out loud without notes; anything you cannot explain is the next thing to study.
- Use the interview bank for practice: answer first, then compare. The green "In DataPulse" lines are your hooks for saying "in my project, I...".
- Source documents: `docs/architecture/DataPulse_Technical_Project_Report_v1.pdf` and `docs/architecture/DataPulse_System_Architecture_Article.pdf`.

> **Note on interview content.** The questions are general senior backend and full-stack questions that fit this stack. They are not taken from any specific company's interview process.

Contents

1. Where we are today 3

What already works . . . 3

What the code review found . . . 3

2. The build phases 4

Phase 0: Foundation fixes . . . 4

Phase 1: Design the contracts (Milestone 2) . . . 4

Phase 2: Domain model, auth and tenant isolation . . . 5

Phase 3: Ingestion API and synthetic data (Milestone 3) . . . 5

Phase 4: Processing and storage (Milestone 4) . . . 6

Phase 5 and beyond: CI, frontend, observability, load testing . . . 6

3. Open decisions 8

Decision 1: How ingestion clients authenticate . . . 8

Decision 2: Where a batch waits before processing . . . 8

4. Senior full-stack interview bank 9

A. System design and architecture . . . 9

B. API design, Django and DRF . . . 10

C. Python . . . 11

D. PostgreSQL . . . 11

E. Celery, Redis and asynchronous processing . . . 12

F. ClickHouse . . . 13

G. Elasticsearch . . . 14

H. Frontend: React, Next.js and TypeScript . . . 14

I. Security and multi-tenancy . . . 15

J. Observability and reliability . . . 16

K. Testing, CI/CD and DevOps . . . 16

L. Scaling and performance . . . 17

M. Senior judgement . . . 17

5. Glossary 18

1. Where we are today

DataPulse is at the end of Milestone 1 (requirements) and the start of Milestone 2 (architecture). The skeleton exists and runs; no product features are built yet.

What already works

- **Django 5.2 project** in `backend/config/`, configured from environment variables via django-environ. `manage.py check` passes.
- **Celery** app wired to Redis with a sample task, `add_numbers`, to prove the worker runs.
- **Docker Compose** with PostgreSQL 17, Redis 7, ClickHouse 25.3 and Elasticsearch 8.17, each with a health check and a persistent volume.
- **Design documents**: a technical project report (requirements, targets, acceptance criteria) and a system architecture article, plus an architecture infographic.
- **README** rewritten with accurate setup steps (12 October 2026).

What the code review found

Issue	Why it matters	Fix
`DJANGO_DEBUG=false` still enables debug mode. The cast is declared for `DEBUG` but the code reads `DJANGO_DEBUG`, so the value stays the string "false", which is truthy.	Debug mode in production leaks stack traces, settings and SQL to anyone who triggers an error.	`environ.Env(DJANGO_DEBUG=(bool, False))`
`.env.example` uses Docker hostnames (`postgres`, port 5432) but Django runs on the host.	A new developer following the README cannot connect to the database.	Use `localhost:5436` and `redis://localhost:6379`.
Redis, ClickHouse and Elasticsearch are published on all network interfaces; Redis and Elasticsearch have no authentication.	Anyone on the same network can read or wipe the data.	Publish as `127.0.0.1:port:port`.
ClickHouse health check logs in as the `default` user.	The container may report unhealthy even when it is fine.	Pass `--user` and `--password`.
Most apps are empty folders, `apps/` has no `__init__.py`, and only `processing` is in `INSTALLED_APPS` (not `rest_framework`).	Models and migrations will not be discovered; the installable package contains no code.	Create proper app packages and register them.
No tests, no CI, no git commits.	No safety net for any change that follows.	Smoke tests, then a first commit.

2. The build phases

Phases follow the milestones in the technical report and are ordered by dependency. Each one is small enough to finish, test and commit before moving on. We decide the details of each phase together before writing code.

Phase 0: Foundation fixes

Goal: A clean, safe, reproducible base that everything else is built on.

TASKS

- Fix the `DEBUG` parsing bug in `config/settings.py`.
- Correct `.env.example` for running Django on the host.
- Bind all Compose ports to `127.0.0.1`, drop ClickHouse port 9000, fix the ClickHouse health check.
- Turn every app into a proper package (`__init__.py`, `apps.py`), add `apps/__init__.py`, and register `rest_framework` and all apps.
- Run `ruff format`, add `conftest.py`, add smoke tests for settings and the Celery task.
- Make the first git commit.

WHY

Every later phase inherits these settings. A configuration bug that ships to production is a security incident, and a setup that only works on one laptop fails the report's "documented local startup" acceptance criterion.

CONCEPTS TO MASTER

- Twelve-factor configuration: config in the environment, typed and validated at startup.
- Docker networking: container DNS names vs published host ports.
- Liveness vs readiness: a running process is not necessarily ready for work.
- Django's app registry and how models and migrations are discovered.
- Python regular packages vs namespace packages.

> **Done when:** A fresh clone, following only the README, reaches healthy containers, a passing `manage.py check` and green tests.

Phase 1: Design the contracts (Milestone 2)

Goal: Pin down the decisions that are expensive to change once code and clients depend on them.

TASKS

- `docs/event-contract.md`: required and optional fields with types, `event_id` format, UTC ISO 8601 timestamps, size limits per event, per batch and for `attributes`, and the rules for evolving `schema_version`. Include examples of valid and invalid events.
- `docs/job-lifecycle.md`: the job state machine (queued, processing, completed, partially_failed, failed), the allowed transitions, per-destination status for ClickHouse and Elasticsearch, and the retry policy.
- `docs/data-model.md`: PostgreSQL entities (User, Organization, Project, Membership, ApiKey, IngestionJob, AuditLog) and an entity-relationship diagram.

WHY

The architecture article says these must be defined before implementation. The event contract is a public API; job states drive the API, the UI and the retry logic.

CONCEPTS TO MASTER

- Schema evolution: backward and forward compatibility, additive changes.
- Time-ordered identifiers (UUIDv7, ULID) and why they index better than random UUIDv4.
- Event time vs ingestion time, and late or out-of-order events.

- Finite state machines and enforcing valid transitions.

> **Done when:** All three documents written, reviewed together and committed.

Phase 2: Domain model, auth and tenant isolation

Goal: Users, organizations and projects, with access control that provably keeps tenants apart.

TASKS

- Custom `User` model, created before the first real migration.
- `Organization`, `Project` and `Membership` (with role) models.
- Project API keys: random secret with a visible prefix, stored as a hash, shown once, revocable.
- DRF authentication class for API keys; permission classes; a base mixin that scopes every queryset to the caller's projects.
- `/api/v1/` routing, `/health` (liveness) and `/ready` (checks Postgres and Redis).
- Cross-tenant tests: organization A must get 404 for organization B's resources.

WHY

Cross-tenant data leaks are the most damaging bug in a multi-tenant product, and the report makes "cross-project access is denied" an acceptance criterion. Swapping the User model later is painful.

CONCEPTS TO MASTER

- Authentication vs authorization; role-based access control.
- IDOR (insecure direct object reference) and why 404 beats 403.
- Hashing high-entropy API keys (SHA-256) vs passwords (Argon2, PBKDF2).
- The DRF request pipeline: authentication, permissions, throttling.
- Defense in depth, including optional Postgres row-level security.

> **Done when:** Every endpoint has a cross-tenant test; no key is stored in plain text.

Phase 3: Ingestion API and synthetic data (Milestone 3)

Goal: Clients can submit events and track what happened to them.

TASKS

- Synthetic generator (management command): configurable volume, event types, latency distributions, malformed events and duplicates.
- `POST /api/v1/ingest/`: validate against the contract, enforce size limits, create an `IngestionJob` and stage the batch in one transaction, enqueue the Celery task with `transaction.on_commit`, return `202` with the job ID and any per-event errors.
- `GET /api/v1/jobs/{id}/` for job status.
- Throttling backed by Redis; optional `Idempotency-Key` header.

WHY

Accepting work quickly and processing it later protects the p95 < 300 ms API target. Enqueueing on commit prevents the worker from starting before the job row exists.

CONCEPTS TO MASTER

- Asynchronous APIs: `202 Accepted`, polling, job resources.
- The dual-write problem, `on_commit` and the transactional outbox pattern.
- Passing references, not payloads, through a queue.
- Partial acceptance vs all-or-nothing batches.
- Rate-limiting algorithms: fixed window, sliding window, token bucket.

Done when: The generator pushes 100,000 events; invalid events are reported clearly; job status is visible; API p95 is measured and recorded.

Phase 4: Processing and storage (Milestone 4)

Goal: Events reliably reach ClickHouse and Elasticsearch exactly once in effect, even with retries and crashes.

TASKS

- Celery task: load the staged batch, normalize, deduplicate on `(project_id, event_id)`, write to each store.
- Reliability settings: `acks_late`, `reject_on_worker_lost`, prefetch multiplier 1, retries with exponential backoff and jitter, bounded `max_retries`.
- A periodic reaper (Celery beat) that re-queues jobs stuck in processing.
- ClickHouse `events` table: `ReplacingMergeTree`, monthly partitions, ordering key starting with `project_id`, TTL for retention, batched inserts.
- Elasticsearch index template with explicit mappings; `_id = event_id`; bulk indexing with per-item error handling.
- Track ClickHouse and Elasticsearch outcomes separately so a failure in one is retried on its own.
- Integration tests: duplicate delivery, worker killed mid-task, Elasticsearch down.

WHY

Celery with acks_late delivers at least once, so duplicates will happen. Correct counts must come from idempotent writes, not from hoping each task runs once.

CONCEPTS TO MASTER

- At-least-once vs exactly-once delivery; idempotency.
- Redis broker visibility timeout and redelivery.
- Poison messages and dead-letter handling.
- ClickHouse parts, merges, sparse primary index; why small inserts hurt; deduplication with `FINAL` or `argMax`.
- Elasticsearch inverted index, `text` vs `keyword`, refresh interval and near-real-time search.
- Partial failure across independent stores and reconciliation.

Done when: Replaying a batch does not change counts; killing a worker loses no job; an Elasticsearch outage is recovered by retrying only the search write.

Phase 5 and beyond: CI, frontend, observability, load testing

Goal: Milestones 5 to 7: a usable product that is observable and measured.

TASKS

- GitHub Actions: ruff, mypy, pytest with service containers, Docker image build. (Can start right after Phase 0.)
- Next.js frontend: dashboard, event explorer, job status view; TypeScript types generated from the OpenAPI schema.
- Observability: Sentry first; then Prometheus metrics, Grafana dashboards, Loki logs and Tempo traces via OpenTelemetry.
- Load tests (k6 or Locust): 1 million events, 500 events/s for 15 minutes, 1,000 events/s bursts, 30-minute memory soak, failure injection.

WHY

The report's definition of success is measured performance and diagnosable failures, not just working features.

CONCEPTS TO MASTER

- React Server vs Client Components; data fetching and caching.
- Cursor pagination and list virtualization.
- RED and USE metrics; percentiles and histograms; SLOs and burn-rate alerts.
- Trace context propagation across HTTP and Celery.
- Backpressure and capacity planning.

Done when: Every acceptance criterion in the report has evidence: tests, dashboards or a benchmark report.

3. Open decisions

These need an answer before Phase 2 and Phase 3. We decide them together.

Decision 1: How ingestion clients authenticate

Option	Pros	Cons
Per-project API keys	Simple for clients; scoped to one project; easy to revoke and rotate.	Long-lived secrets; must be hashed and protected.
Short-lived JWTs from a service account	Tokens expire on their own; standard format.	Clients need a token exchange step; revocation is harder.
OAuth 2 client credentials	Industry standard for machine-to-machine.	Most setup for a project of this size.

Recommendation: per-project API keys for ingestion; Django session cookies (httpOnly, SameSite, with CSRF protection) for the dashboard.

Decision 2: Where a batch waits before processing

Option	Pros	Cons
PostgreSQL table (JSONB)	Durable; written in the same transaction as the job row; no new service.	Adds write load and storage to Postgres; needs cleanup.
Inside the Redis message	Simplest code.	Not durable; large messages; the architecture doc advises against it.
Object storage (S3 or MinIO)	Scales to large batches; cheap.	Another service; the upload and the job row are not atomic.

Recommendation: a PostgreSQL staging table, deleted after successful processing. Move to object storage if batch sizes or volumes outgrow it.

4. Senior full-stack interview bank

72 questions across every part of the system, pitched at senior level. Answers are deliberately compact: they give the core idea, the trade-off and the detail that shows depth. Practise answering aloud before reading the answer.

> **Answering like a senior engineer:** state the trade-off, not just the technique; say how you would measure; name the failure mode; tie it to something you built.

A. System design and architecture

Q1. Walk me through the life of an event in DataPulse.

The client posts a batch with a project API key. DRF authenticates the key, resolves the project, applies throttling and validates each event against the versioned schema. In one Postgres transaction we create an `IngestionJob` (queued) and stage the batch; `transaction.on_commit` enqueues a Celery task carrying only the job ID. The API returns `202` with the job ID in tens of milliseconds. A worker loads the batch, normalizes, deduplicates by `(project_id, event_id)`, bulk-inserts into ClickHouse and bulk-indexes into Elasticsearch, recording each destination's outcome, then moves the job to completed, partially_failed or failed. The dashboard reads aggregates from ClickHouse, search from Elasticsearch and job status from Postgres, always through the API.

Q2. Why use three databases instead of one?

The workloads differ. Postgres is row-oriented OLTP with transactions and constraints, ideal for users, projects and job state. ClickHouse is columnar OLAP: it scans millions of rows for counts and percentiles in milliseconds but is poor at single-row updates. Elasticsearch is an inverted index for full-text search and ad-hoc filters. The cost is three write paths, eventual consistency and more operations work. A smaller product could start on Postgres alone (partitioning, BRIN indexes, `pg_trgm`) and split out stores when measurements show the need.

In DataPulse: The split mirrors the access patterns in the report: dashboards, search and job tracking.

Q3. Why ingest asynchronously instead of writing to ClickHouse inside the request?

It decouples API latency and availability from downstream stores, absorbs bursts in the queue, lets the system retry without involving the client, and lets workers build the large batches ClickHouse prefers. The trade-offs are eventual consistency, the need to track jobs, and a queue that can grow without bound unless you monitor backlog and apply backpressure.

Q4. What consistency guarantees does the system give?

Postgres is strongly consistent per transaction, and acceptance is durable once `202` is returned because the batch is committed. Analytics and search are eventually consistent, with a target of 95% of events visible in ClickHouse within ten seconds. The two stores can briefly disagree, so we track per-destination status and show it in the UI. We do not claim exactly-once delivery; at-least-once delivery plus idempotent writes gives effectively-once results.

Q5. How would the design change to handle a billion events a day?

Measure first, then: scale the stateless API horizontally; consider replacing Celery and Redis with Kafka or Redpanda for a durable, partitioned, replayable log; shard and replicate ClickHouse (`ReplicatedMergeTree` with `Distributed` tables), add TTL retention, materialized-view rollups and tiered storage; use time-based Elasticsearch data streams with lifecycle policies and sensible shard sizes; keep Postgres small (metadata only) behind PgBouncer; move batch staging to object storage.

Q6. Where are the single points of failure, and how do you mitigate them?

Locally: one Redis, one Postgres, one ClickHouse and one Elasticsearch node. If Redis loses in-flight messages, tasks vanish; because job state lives in Postgres, a reaper can find stuck jobs and re-enqueue them. In production: managed Postgres with failover, Redis with persistence and Sentinel (or a durable broker), and replicated ClickHouse and Elasticsearch clusters. The key principle is that Redis is never the source of truth.

Q7. How do you evolve the event schema without breaking clients?

Every event carries `schema_version`. Additive, optional fields are backward compatible. A breaking change gets a new version; the API accepts old and new during a deprecation window and workers upcast old versions to the internal model. In ClickHouse, `ADD COLUMN` is a cheap metadata change and free-form data lives in a `Map` or JSON column. In Elasticsearch, changing a field's type requires a reindex into a new index behind an alias.

B. API design, Django and DRF

Q8. Why return 202 Accepted rather than 201 Created for ingestion?

201 says the resource exists and is complete. Here, processing has not happened yet. 202 says the request is accepted for processing; the body includes the job ID and a URL to poll, plus counts of accepted and rejected events.

Q9. Should one invalid event cause the whole batch to be rejected?

It is a trade-off. All-or-nothing is simpler to reason about; partial acceptance avoids dropping 99 good events because of one bad one. For telemetry I would accept valid events and return per-index errors, while batch-level problems (bad auth, oversize body, malformed JSON) reject the whole request with a 4xx. Whichever you choose must be explicit in the contract.

Q10. Describe the DRF request lifecycle.

Django middleware runs, the URL resolver picks the view, and `APIView.dispatch` calls `initial()`: authentication classes set `request.user` and `request.auth`, then permission checks, then throttles. The handler runs, usually validating with a serializer, and the response is rendered by content negotiation. Exceptions go through the exception handler. Object-level permissions run when `get_object()` calls `check_object_permissions`.

Q11. How do you find and fix N+1 queries?

N+1 happens when serializing a list triggers a query per row for related data. Use `select_related` for foreign keys (a SQL join) and `prefetch_related` for reverse and many-to-many relations (a second query joined in Python), `Prefetch` objects for filtered prefetches, and `annotate` instead of per-row counts. Detect with django-debug-toolbar or nplusone, and lock the fix in with `assertNumQueries` in tests.

Q12. How do you paginate very large result sets?

Offset pagination slows down as the offset grows, because the database reads and discards the skipped rows, and pages shift when new rows arrive. Use keyset (cursor) pagination: `WHERE (ts, id) < (:last_ts, :last_id) ORDER BY ts DESC, id DESC LIMIT n`, which uses an index and is stable. DRF provides `CursorPagination`. In Elasticsearch, use `search_after` with a point-in-time, since deep `from`/`size` is capped at 10,000 by default.

Q13. How do you version a public API?

URL versioning (`/api/v1/`) is explicit and easy for clients; header versioning is cleaner but less visible. Within a version, make only additive changes; deprecate with headers and a published timeline. The API version and the event `schema_version` are independent.

Q14. How do you make the ingestion endpoint idempotent?

Accept an `Idempotency-Key` header. Store the key, a hash of the request and the response, with a unique constraint on `(project_id, key)`. A retry with the same key returns the stored response; the same key with a different body returns 409 or 422. Event-level deduplication by `event_id` still happens downstream, so there are two layers of protection.

Q15. Where should business logic live in a Django project?

Keep views thin (HTTP concerns only) and serializers focused on validation and shape. Put business operations, such as creating an ingestion job or transitioning its state, in a service layer so views, Celery tasks and management commands reuse them and unit tests are simple. Models hold invariants close to the data: constraints and state-transition methods.

C. Python

Q16. How does the GIL affect Django and Celery?

In standard CPython, the GIL lets only one thread run Python bytecode at a time per process (3.13 added an optional free-threaded build). I/O releases the GIL, so threads help I/O-bound work; CPU-bound work needs processes. Gunicorn and Celery's default prefork pool both use processes. For workloads dominated by network I/O, gevent or asyncio help. Event normalization is light CPU and the writes are I/O, so tuning concurrency matters more than the GIL.

Q17. When would you use async Django?

When a request awaits several slow I/O operations that can run concurrently, such as fanning out to ClickHouse and Elasticsearch, or for long-lived connections. Mixing sync middleware or ORM calls adds thread hops. For ingestion, the work is short and handed to Celery, so sync WSGI with enough workers is simpler. Measure before switching.

Q18. How do you process a large batch without running out of memory?

Stream instead of loading everything: generators yield items lazily, you normalize as you go and write in fixed-size chunks. In Django, `.iterator(chunk_size=...)` streams query results instead of caching them all. This is how you meet a "no unbounded memory growth" requirement.

Q19. How do you type and validate data across the boundary?

Validate at the edge with DRF serializers (or Pydantic), use dataclasses or Pydantic models internally, and run mypy or pyright in CI. Generate an OpenAPI schema with drf-spectacular and generate TypeScript types from it, so a backend change breaks the frontend build rather than production.

D. PostgreSQL

Q20. How do you choose indexes for the ingestion jobs table?

From the actual queries. Listing a project's newest jobs needs a composite index on `(project_id, created_at DESC)`. Finding stuck jobs suits a partial index on `updated_at WHERE status IN ('queued', 'processing')`, which stays small. Idempotency needs a unique index on `(project_id, idempotency_key)`. Verify with `EXPLAIN (ANALYZE, BUFFERS)`, remember every index slows writes, and check `pg_stat_user_indexes` for unused ones.

Q21. Explain MVCC and why it matters for a job-status table.

Postgres keeps multiple row versions so readers do not block writers. An update writes a new version and leaves a dead one for VACUUM. Frequently updated status rows cause bloat; tune autovacuum and set `fillfactor` below 100 to allow HOT updates (possible when no indexed column changes). Long-running transactions stop vacuum from cleaning up, so workers must keep transactions short.

Q22. How do several workers safely claim jobs from a table?

Use `SELECT ... FOR UPDATE SKIP LOCKED` so each worker locks a different row without waiting, or a conditional update such as `UPDATE jobs SET status='processing' WHERE id = :id AND status = 'queued'` and check the row count. Never check and then act in separate statements; two workers can both see "queued".

Q23. Which isolation level would you use, and why?

The default, Read Committed, gives each statement a fresh snapshot. Repeatable Read gives a snapshot for the whole transaction. Serializable detects anomalies but you must retry on serialization failures. For job transitions, Read Committed with conditional updates or row locks is simpler and sufficient.

Q24. How do you manage database connections with Django and Celery?

Each process holds its own connections, so many Gunicorn workers plus Celery processes can exceed `max_connections`. Use PgBouncer in transaction mode (watch out for session features such as server-side cursors) or Django 5.1+'s native psycopg pool. Celery's Django integration closes stale connections around tasks; `CONN_MAX_AGE` controls persistence.

Q25. How would you partition a large audit log table?

Declarative range partitioning by month on `created_at`. Queries that filter on time skip irrelevant partitions, and retention becomes `DROP` or `DETACH` of an old partition, which is instant and avoids huge `DELETE`s and vacuum work. Automate partition creation with pg_partman or a scheduled task.

Q26. When is JSONB the right choice?

For flexible, sparse or opaque data, such as staged raw batches or per-project settings, optionally with a GIN index for containment queries. Not for fields you join on, filter constantly or need constraints on; those deserve real columns. Large values are compressed and stored out of line (TOAST).

E. Celery, Redis and asynchronous processing

Q27. What happens if a worker crashes halfway through a task?

By default Celery acknowledges a message when the worker receives it, so a crash loses it. With `task_acks_late=True` the ack happens after the task finishes, so the broker redelivers. With Redis, unacknowledged messages come back after the `visibility_timeout` (one hour by default), which must exceed your longest task or tasks run twice. `task_reject_on_worker_lost=True` requeues when the child process is killed. Redelivery means at-least-once, so tasks must be idempotent. A periodic reaper re-enqueues jobs stuck in processing as a safety net.

Q28. How do you make a Celery task idempotent?

Pass the job ID, not the data. Start with an atomic conditional transition (queued to processing); if the job is already complete, exit. Make every write keyed: Elasticsearch documents use `_id = event_id`, so a second write overwrites; ClickHouse uses `ReplacingMergeTree` to collapse duplicates. Record per-destination status so a retry skips stores that already succeeded.

> In DataPulse: This is the core reliability design of Phase 4.

Q29. What is your retry strategy?

Retry only transient errors (connection failures, timeouts) using `autoretry_for`, with `retry_backoff=True` for exponential backoff, `retry_jitter=True` to avoid synchronized retry storms, a `retry_backoff_max` cap and a bounded `max_retries`. Validation errors fail immediately. When retries are exhausted, mark the job failed with the error, emit a metric and make it reprocessable.

Q30. What does `worker_prefetch_multiplier` do, and what would you set it to?

It controls how many messages each worker process reserves in advance. High values help throughput for many tiny tasks but distribute work unevenly and hold messages hostage behind long tasks. For long batch tasks with late acks, use 1.

Q31. How do you implement backpressure?

Measure queue depth and processing lag. Past a threshold, the API returns 429 or 503 with `Retry-After` instead of accepting unlimited work. Bound worker concurrency and batch sizes, and use separate queues per workload (ingestion vs reindexing) so one cannot starve the other.

Q32. How does Redis persistence work, and does it matter here?

RDB takes periodic snapshots and can lose minutes of writes; AOF logs every write, and with `appendfsync everysec` loses at most about a second. Compose runs Redis with AOF. It matters less than you might think, because Postgres, not Redis, is the source of truth for jobs.

Q33. When would you move from Celery and Redis to Kafka?

When you need a durable, replayable log (reprocess the last seven days), ordered partitions, several independent consumers each with its own offset (ClickHouse sink, Elasticsearch sink, alerting), or very high throughput. A task queue forgets a message once it is consumed. Kafka adds real operational complexity, so the requirements must justify it.

F. ClickHouse

Q34. Why is ClickHouse fast for analytics?

It stores data by column, so a query reads only the columns it uses; similar values compress extremely well; execution is vectorized over column blocks; and a sparse primary index on the sorting key skips whole granules (8,192 rows by default) instead of indexing every row. Queries run in parallel across cores. The price is slow point updates and deletes and a strong preference for large batch inserts.

Q35. Design the events table.

`ReplacingMergeTree(ingested_at)` for deduplication; `PARTITION BY toYYYYMM(timestamp)`, which is coarse enough to avoid too many partitions and makes retention easy; `ORDER BY (project_id, event_type, timestamp, event_id)`, matching tenant and time filters; `LowCardinality(String)` for service, severity and type; a `Map` or JSON column for attributes; `TTL timestamp + INTERVAL 90 DAY`.

> In DataPulse: Ordering by project first makes every tenant-scoped query read a contiguous range.

Q36. Why must you avoid many small inserts?

Each insert creates a new data part on disk that background merges must combine. Thousands of tiny inserts cause merge pressure and "too many parts" errors. Batch thousands to hundreds of thousands of rows per insert, or enable `async_insert` so the server buffers for you.

Q37. What are the gotchas of ReplacingMergeTree?

Deduplication happens only during merges, at an unpredictable time, and only within a partition, so queries can see duplicates. Options: `SELECT ... FINAL` (correct but slower), aggregating with `argMax` or `uniqExact(event_id)`, or `OPTIMIZE ... FINAL` (expensive; not for routine use). Tests must prove counts are correct while duplicates are still present.

Q38. How do you make dashboard queries fast?

Pre-aggregate on insert with materialized views into `SummingMergeTree` or `AggregatingMergeTree` tables (for example events per minute by project, type and severity) and query the rollup for long ranges. Use projections for alternative sort orders, always bound the time range, and limit result size.

G. Elasticsearch

Q39. How does Elasticsearch search work?

It builds an inverted index mapping terms to documents. `text` fields are analyzed (tokenized, lowercased, optionally stemmed) for full-text search; `keyword` fields are stored as-is for exact matches, aggregations and sorting. An index is split into shards, each a Lucene index, with replicas. A query fans out to every shard, each returns its top hits, and the coordinating node merges them. Relevance scoring uses BM25.

Q40. How would you map the events index?

Use an explicit index template. Map `message` as `text`; `service`, `severity`, `event_type`, `project_id` and `trace_id` as `keyword`; `timestamp` as `date`. Map free-form attributes as `flattened` to prevent a mapping explosion. Put structured filters in filter context, which skips scoring and is cacheable, and use query context only for text.

Q41. What is near-real-time search, and how does it affect ingestion?

New documents become searchable after a refresh (every second by default) creates a new segment. Frequent refreshes create many small segments to merge. For heavy ingestion, lengthen `refresh_interval` and accept slower search visibility, measured separately. Use the bulk API, and check its `errors` flag: a bulk request returns 200 even when individual items fail.

Q42. How do you enforce tenant isolation in Elasticsearch?

The server always builds the query and wraps it in a `project_id` filter; clients never send raw query DSL. Alternatives include an index per tenant (strong isolation but too many shards for many small tenants), filtered aliases or document-level security. With many small tenants, a shared index with a mandatory filter and custom routing by `project_id` works well.

Q43. How do you change a field's mapping in production?

You cannot change an existing field's type in place. Create a new index with the new mapping, reindex (or replay from the source of truth), dual-write during the switch, then atomically move the alias. That is why applications should only ever use alias names.

H. Frontend: React, Next.js and TypeScript

Q44. How would you structure the Next.js app?

App Router with routes such as `/projects/[id]/dashboard`, `/events` and `/jobs`. Server Components fetch data on the server, ship no JavaScript and keep secrets server-side; Client Components handle interactivity such as charts and filters. A typed API client generated from the OpenAPI schema, and TanStack Query for client-side caching and polling.

Q45. What is the difference between Server and Client Components?

Server Components render on the server, can be async and fetch directly, ship zero JavaScript, and cannot use state, effects or browser APIs. Client Components (`'use client'`) are hydrated in the browser and are interactive. Props passed from server to client must be serializable. Keep the client boundary as low in the tree as possible.

Q46. How do you chart a million events in the browser?

You don't send a million events. The backend aggregates in ClickHouse into time buckets sized to the range (a few hundred points). The frontend uses canvas-based charts when there are many series, debounces filter changes, and cancels stale requests with `AbortController` so an older response cannot overwrite a newer one.

Q47. How do you build an event explorer over a huge result set?

Cursor pagination (`search_after`), a virtualized list that renders only visible rows, debounced search input, filter state kept in the URL so views are shareable and the back button works, skeleton loading states, and a clear message that new events can take a few seconds to appear.

Q48. How would you show live job progress?

Polling (simple and robust at a 2 to 5 second interval, stopping on a terminal status), Server-Sent Events (a one-way stream over HTTP with automatic reconnection), or WebSockets (two-way, but needs Django Channels and ASGI). For job status, polling or SSE is enough.

Q49. Where should the frontend keep auth tokens?

In httpOnly, Secure, SameSite cookies, which JavaScript cannot read, so an XSS bug cannot steal them; pair them with CSRF protection. Tokens in `localStorage` are readable by any injected script. Next.js middleware can guard routes, but the API is where authorization is truly enforced.

Q50. How do you diagnose and fix slow re-renders in a dashboard?

Measure with the React Profiler first. Then keep state close to where it is used, split contexts that change often, keep references stable (`useMemo`, `useCallback`, or the React Compiler), virtualize long lists, and move heavy computation to the server or a web worker.

I. Security and multi-tenancy

Q51. How do you guarantee one organization cannot see another's data?

Defense in depth: a base mixin scopes every queryset to the caller's projects, never `Model.objects.all()`; object lookups go through that scoped queryset so foreign IDs return 404 and do not reveal existence; ClickHouse and Elasticsearch queries get a mandatory server-side `project_id` filter; Celery tasks carry and re-check `project_id`; every endpoint has a cross-tenant test. Postgres row-level security can act as a final backstop.

> In DataPulse: This is an acceptance criterion in the technical report.

Q52. How should API keys be stored?

Generate a high-entropy secret (`secrets.token_urlsafe(32)`) with a recognizable prefix, show it once, and store the prefix for lookup plus a SHA-256 hash. A fast hash is fine because the key is random; passwords need slow hashes (Argon2, PBKDF2) because humans choose weak ones. Compare in constant time, and support scopes, rotation, revocation and `last_used_at`.

Q53. Which OWASP risks apply most to DataPulse?

Broken access control (tenant isolation, IDOR), injection (ClickHouse SQL and Elasticsearch DSL built from user input), mass assignment (always use explicit serializer fields), excessive data exposure, missing rate limits, SSRF once webhooks exist (validate URLs and block internal ranges), and secrets leaking into logs.

Q54. How do you prevent injection in dynamic ClickHouse queries?

Use the driver's parameter binding (clickhouse-connect supports typed server-side parameters such as `{project_id:UUID}`). For dynamic identifiers, such as a group-by column, map user input through an allow-list of known column names. Never interpolate user input into SQL.

Q55. How do you handle retention and deletion across three stores?

Retention: ClickHouse TTL, Elasticsearch lifecycle policies or dropping old indices, Postgres partition drops. A deletion request becomes a tracked job: Elasticsearch `delete_by_query` (or drop a tenant index), ClickHouse lightweight `DELETE` or a mutation, Postgres deletes, each confirmed before the job completes. Best of all, avoid storing personal data in events at all.

J. Observability and reliability

Q56. When do you use metrics, logs and traces?

Metrics are cheap aggregates for trends and alerts (request rate, p95 latency, queue depth). Logs are detailed records for debugging a specific failure. Traces show causality and timing across services, for example API to queue to worker to ClickHouse. Link them with a shared trace ID in logs and exemplars on metrics.

Q57. What would you alert on?

Symptoms users feel, tied to SLOs: ingestion error rate, API p95 latency, time until events are visible in analytics, a queue backlog that keeps growing, failed or stuck jobs. Use error-budget burn-rate alerts. Avoid paging on causes such as CPU alone.

Q58. Why report percentiles rather than averages, and can you average percentiles?

Averages hide the tail; p95 and p99 describe the worst experiences. Percentiles cannot be averaged across instances; record histograms, aggregate the buckets, then compute the quantile (`histogram_quantile` in Prometheus).

> In DataPulse: The report's measurement rules require percentiles from measured requests.

Q59. How do you trace a request across a Celery task?

OpenTelemetry instrumentation for Django and Celery propagates the W3C `traceparent` in task headers, so the worker's spans join the same trace. Storing the trace ID on the `IngestionJob` lets you jump from a job straight to its trace.

Q60. What are RED and USE?

RED (rate, errors, duration) describes services and endpoints. USE (utilization, saturation, errors) describes resources such as CPU, memory, connection pools and queues. A good service dashboard shows both.

K. Testing, CI/CD and DevOps

Q61. What is your testing strategy?

Many fast unit tests for validation, normalization and state transitions. Integration tests against real Postgres, Redis, ClickHouse and Elasticsearch in containers, because mocks hide real behavior such as delayed deduplication. API tests for auth and tenant isolation, contract tests for the event schema, a few end-to-end tests for critical UI flows, and separate load and failure-injection tests.

Q62. How do you test Celery tasks?

Unit-test the task's logic by calling it directly. Avoid relying on eager mode for integration tests, because it skips serialization and changes `on_commit` timing; use a real worker (Celery's pytest worker fixture). Test idempotency by running the task twice and asserting counts do not change.

Q63. Describe the CI pipeline you would build.

On every pull request: ruff, mypy and pytest with service containers and a coverage threshold; for the frontend, `tsc --noEmit`, ESLint, unit tests and a production build. Cache dependencies. On main: build images tagged with the git SHA, scan them, push, deploy to staging, smoke-test, then promote to production.

Q64. How do you run database migrations with zero downtime?

Expand and contract: add a nullable column, deploy code that writes both, backfill in batches, switch reads, drop the old column in a later release. Avoid long locks: build indexes concurrently (`AddIndexConcurrently` in a non-atomic migration) and add constraints as `NOT VALID`, then `VALIDATE`. Never rename a column in one step while old code is running.

Q65. What makes a good production Dockerfile for Django?

Multi-stage build; pinned slim base image; dependencies installed before code is copied, for layer caching; a non-root user; a `.dockerignore`; Gunicorn as the entry point; no secrets baked in. Use the same image for the API and the worker with different commands.

L. Scaling and performance

Q66. API p95 latency is 800 ms against a 300 ms target. How do you investigate?

Reproduce under a fixed load, then use traces to see where time goes. Usual suspects: N+1 queries, a missing index, an uncached API-key lookup, opening new database connections per request, slow enqueue, or large JSON parsing (orjson helps). Change one thing at a time, re-measure and record the result.

Q67. Where would you add caching, and how would you invalidate it?

Cache the API-key-to-project lookup with a short TTL, invalidated on revocation. Cache dashboard aggregates keyed by project, query and time bucket: short TTLs for recent buckets, long for historical ones. Use cache-aside, protect against stampedes with a lock or stale-while-revalidate, and always include the tenant in the cache key.

Q68. How would you load-test 500 events per second sustained?

Drive the synthetic generator from k6 or Locust; record hardware, batch size and concurrency. Ramp up, sustain for 15 minutes, burst to 1,000 events per second for 60 seconds, and run a 30-minute soak. Measure client-side p95, server metrics, queue depth, time to visibility and memory, then inject failures mid-test.

> In DataPulse: These are exactly the targets in the technical report.

Q69. What is the difference between scaling up and scaling out here?

Scaling up (a bigger machine) is simplest and often right for Postgres and early ClickHouse. Scaling out works naturally for stateless parts (the API and workers), but stateful stores need sharding and replication, which add operational and consistency costs. Scale out the stateless tier first.

M. Senior judgement

Q70. What was the biggest trade-off you made in this project?

For example: three specialized stores instead of one, trading consistency and operational cost for query performance; or at-least-once delivery with idempotent writes instead of chasing exactly-once. Explain the reasoning, what you measured and what would make you revisit it.

Q71. How would you onboard a junior engineer to this codebase?

A README that gets them running in minutes, the architecture documents, written architecture decision records explaining why things are the way they are, a well-scoped first issue, pairing, and code reviews that explain the reasoning rather than just requesting changes.

Q72. What would you do differently building this for a real company?

Use managed services (managed Postgres, ClickHouse Cloud, Elastic Cloud) to cut operations work; define SLOs from day one; record decisions as ADRs; add feature flags, a security review and cost monitoring; consider Kafka only if the scale requires it.

5. Glossary

Term	Meaning
At-least-once delivery	Every message is delivered one or more times; duplicates are possible, loss is not.
Idempotent	Doing an operation twice has the same effect as doing it once.
Eventual consistency	Stores may briefly disagree but converge once processing catches up.
Backpressure	Slowing or refusing new work when the system is falling behind.
Dead-letter queue	Where messages go after exhausting retries, for inspection or replay.
Visibility timeout	How long a broker waits for an ack before redelivering a message.
OLTP / OLAP	Transactional workloads (many small reads and writes) vs analytical workloads (large scans and aggregations).
Columnar storage	Storing each column together, so queries read only the columns they need.
MergeTree	ClickHouse's main table engine family: sorted parts merged in the background.
Inverted index	A map from each term to the documents that contain it; the basis of full-text search.
Shard / replica	A slice of the data / a copy of a slice for redundancy and read capacity.
MVCC	Multi-version concurrency control: readers see a snapshot and do not block writers.
Keyset pagination	Paging with `WHERE key < last_key` instead of `OFFSET`.
IDOR	Insecure direct object reference: accessing another tenant's object by guessing its ID.
Tenant isolation	Guaranteeing each organization sees only its own data.
Transactional outbox	Writing a message in the same database transaction as the data, then publishing it, to avoid dual-write inconsistencies.
p95 latency	95% of requests are faster than this value.
SLO / error budget	A reliability target, and the amount of failure it allows before action is needed.
RED / USE	Service metrics (rate, errors, duration) / resource metrics (utilization, saturation, errors).
Liveness / readiness	Is the process alive? / Is it ready to accept work?